

Fine-tuning of the NLLB model for Europarl-ST (Spanish to English)

In this notebook, we are going to explain how to fine-tune the [NLLB model](#) on the [Europarl-ST dataset](#). First, we need to run the experiment L4.1 on the development set to generate data to be used in the fine tuning process. Once the fine tuning of the NLLB is carried out, we will evaluate the NLLB model performance over the Europarl-ST test data where speech transcriptions were obtained also in the L4.1 experiment. Remember that the initial performance of the NLLB model on the test data evaluated in experiment L4.3 was 9.3 of BLEU and 57.9% COMET.

We load the automatic speech transcriptions and reference translations from the csv file generated in experiment L4.1 on the dev data. Moreover, we split the data into train (17%) and validation (test) (83%) sets for fine tuning

```
In [ ]: from datasets import load_dataset

raw_datasets = load_dataset("csv", data_files="L4.1_ASR_Whisper_Baseline_dev_Europarl-ST.csv")

raw_datasets = raw_datasets.select_columns(["hypothesis", "translation"])

raw_datasets = raw_datasets["train"].train_test_split(test_size=0.83, shuffle=False)

print(raw_datasets)
```

Show the training pairs (hypothesis-translation)

```
In [ ]: raw_datasets["train"][:12]["hypothesis"]
```

```
In [ ]: raw_datasets["train"][:12]["translation"]
```

Now we load the pre-trained tokenizer for the NLLB model and apply it to the Spanish-English pair:

```
In [4]: max_tok_length = 275

from transformers import AutoTokenizer

checkpoint = "facebook/nllb-200-distilled-600M"
# from flores200_codes import flores_codes
src_code = "spa_Latn"
tgt_code = "eng_Latn"
tokenizer = AutoTokenizer.from_pretrained(
    checkpoint,
    padding=True,
    pad_to_multiple_of=8,
    src_lang=src_code,
    tgt_lang=tgt_code,
    truncation=False,
    max_length=max_tok_length,
)
```

We can apply the tokenizer function to any dataset taking advantage that Hugging Face Datasets are [Apache Arrow](#) files stored on the disk, so you only keep the samples you ask for loaded in memory.

To keep the data as a dataset, we will use the [Dataset.map\(\) function](#). This also allows us some extra flexibility, if we need more preprocessing done than just tokenization. The map() method works by applying a function on each element of the dataset.

In our case, each sample pair is going to be preprocessed according to the training needs of the model that is to be finetuned:

```
In [5]: def preprocess_function(sample):
        model_inputs = tokenizer(
            sample["hypothesis"],
            text_target = sample["translation"],
        )
        return model_inputs
```

Now, we can apply the preprocess_function to the raw datasets (training, validation and test):

```
In [ ]: tokenized_datasets = raw_datasets.map(preprocess_function, batched=True)
```

We can take a quick look at the length histogram in the source language:

```
In [ ]: dic = {}
for sample in tokenized_datasets['train']:
    sample_length = len(sample['input_ids'])
    if sample_length not in dic:
        dic[sample_length] = 1
    else:
        dic[sample_length] += 1

for i in range(1, max_tok_length+1):
    if i in dic:
        print(f"{i:>2} {dic[i]:>3}")
```

Checking a sample after filtering by maximum number of tokens:

bitsandbytes is a quantization library with a Transformers integration. With this integration, you can quantize a model to 8 or 4-bits and enable many other options by configuring the BitsAndBytesConfig class. For example, you can:

- set load_in_4bit=True to quantize the model to 4-bits when you load it
- set bnb_4bit_quant_type="nf4" to use a special 4-bit data type for weights initialized from a normal distribution
- set bnb_4bit_use_double_quant=True to use a nested quantization scheme to quantize the already quantized weights
- set bnb_4bit_compute_dtype=torch.bfloat16 to use bfloat16 for faster computation

```
In [8]: import torch
from transformers import BitsAndBytesConfig

quantization_config = BitsAndBytesConfig(
    load_in_4bit=True,
    bnb_4bit_quant_type="nf4",
    bnb_4bit_use_double_quant=True,
    bnb_4bit_compute_dtype=torch.bfloat16,
)
```

Pass the quantization_config to the from_pretrained method.

```
In [ ]: from transformers import AutoModelForSeq2SeqLM

model = AutoModelForSeq2SeqLM.from_pretrained(
    checkpoint,
    quantization_config=quantization_config
)
```

Next, you should call the prepare_model_for_kbit_training() function to preprocess the quantized model for training.

```
In [10]: from peft import prepare_model_for_kbit_training

model = prepare_model_for_kbit_training(model, use_gradient_checkpointing=False, gradient_checkpointing_kwargs={'use_
```

[LoRA \(Low-Rank Adaptation of Large Language Models\)](#) is a [parameter-efficient fine-tuning \(PEFT\)](#) technique that significantly reduces the number of trainable parameters. It works by inserting a smaller number of new weights into the model and only these are trained. This makes training with LoRA much faster, memory-efficient, and produces smaller model weights (a few hundred MBs), which are easier to store and share.

Each PEFT method is defined by a `PeftConfig` class that stores all the important parameters for building a `PeftModel`. For example, to train with LoRA, load and create a `LoraConfig` class and specify the following parameters:

- `task_type`: the task to train for (sequence-to-sequence language modeling in this case)
- `r`: the dimension of the low-rank matrices
- `lora_alpha`: the scaling factor for the low-rank matrices
- `target_modules`: determine what set of parameters are adapted
- `lora_dropout`: the dropout probability of the LoRA layers

```
In [11]: from peft import LoraConfig, get_peft_model

config = LoraConfig(
    task_type="SEQ_2_SEQ_LM",
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    bias="none",
)
```

Once LoRA and the quantization are setup, create a quantized `PeftModel` with the `get_peft_model()` function. It takes a quantized model and the `LoraConfig` containing the parameters for how to configure a model for training with LoRA.

```
In [ ]: lora_model = get_peft_model(model, config)
lora_model.print_trainable_parameters()
```

The function that is responsible for putting together samples inside a batch is called a `collate` function. It is an argument you can pass when you build a `DataLoader`, the default being a function that will just convert your samples to PyTorch tensors and concatenate them. This is not possible in our case since the inputs we have are not all of the same size. We have deliberately postponed the padding, to only apply it as necessary on each batch and avoid having over-long inputs with a lot of padding.

To do this in practice, we have to define a `collate` function that will apply the correct amount of padding to the items of the dataset we want to batch together. Fortunately, the `Transformers` library provides us with such a function via `DataCollatorForSeq2Seq` that takes a tokenizer when you instantiate it (to know which padding token to use, and whether the

model expects padding to be on the left or on the right of the inputs), so we will also need to instantiate the model first to provide it to the collate function:

```
In [13]: from transformers import DataCollatorForSeq2Seq
data_collator = DataCollatorForSeq2Seq(
    tokenizer=tokenizer,
    model=lora_model,
    pad_to_multiple_of=8
)
```

Evaluation

The last thing to define for our Seq2SeqTrainer is how to compute the metrics to evaluate the predictions of our model with respect to references. To this purpose, we use the [Evaluate library](#) which includes the definition of generic and task-specific metrics. In our case, we use the [BLEU metric](#), or to be more precise, [sacreBLEU](#). You can see a simple example of usage below:

```
In [14]: from evaluate import load
metric = load("sacrebleu")
```

We need to define a function `compute_metrics` to compute BLEU scores at each epoch. The example below performs a basic post-processing to decode the predictions into texts:

```
In [15]: import numpy as np
def postprocess_text(preds, labels):
    preds = [pred.strip() for pred in preds]
    labels = [[label.strip()] for label in labels]

    return preds, labels

def compute_metrics(eval_preds):
    preds, labels = eval_preds
    if isinstance(preds, tuple):
        preds = preds[0]
    decoded_preds = tokenizer.batch_decode(preds, skip_special_tokens=True)

    # Replace negative ids in the labels as we can't decode them.
    #labels = np.where(labels < 0, labels, tokenizer.pad_token_id)
    for i in range(len(labels)):
        labels[i] = [tokenizer.pad_token_id if j<0 else j for j in labels[i]]
    decoded_labels = tokenizer.batch_decode(labels, skip_special_tokens=True)

    # Some simple post-processing
    decoded_preds, decoded_labels = postprocess_text(decoded_preds, decoded_labels)

    result = metric.compute(predictions=decoded_preds, references=decoded_labels)
    result = {"bleu": result["score"]}

    prediction_lens = [np.count_nonzero(pred != tokenizer.pad_token_id) for pred in preds]
    result["gen_len"] = np.mean(prediction_lens)
    result = {k: round(v, 4) for k, v in result.items()}
    return result
```

Training

The first step before we can define our `Trainer` is to define a `Seq2SeqTrainingArguments` class that will contain all the hyperparameters the Trainer will use for training and evaluation. The only compulsory argument you have to provide is a directory where the trained model will be saved, as well as the checkpoints along the way. For all the rest, you can set them depending on the recommendations from the model developers:

```
In [ ]: from transformers import Seq2SeqTrainingArguments

batch_size = 2
model_name = checkpoint.split("/")[-1]
args = Seq2SeqTrainingArguments(
    f"{model_name}-finetuned-es-to-en",
    evaluation_strategy = "epoch",
    learning_rate=1e-4,
    per_device_train_batch_size=batch_size,
    per_device_eval_batch_size=batch_size,
    weight_decay=0.01,
    save_total_limit=2,
    num_train_epochs=20,
    predict_with_generate=True,
)
```

Once we have our model, we can define a Trainer by passing it all the objects constructed up to now — the model, the `training_args`, the training and validation datasets, the tokenizer, the data collator and the `compute_metrics` function:

```
In [ ]: from transformers import Seq2SeqTrainer

trainer = Seq2SeqTrainer(
    lora_model,
    args,
    train_dataset=tokenized_datasets['train'],
    eval_dataset=tokenized_datasets['test'],
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics
)
```

To fine-tune the model on our dataset, we just have to call the `train()` function of our Trainer:

```
In [ ]: trainer.train()
```

Evaluation of the fine tuned NLLB model on the Europarl-ST test set

Let us first load the default inference parameters of NLLB:

```
In [ ]: from transformers import GenerationConfig

generation_config = GenerationConfig.from_pretrained(
    checkpoint,
)

print(generation_config)
```

We load the speech transcriptions (and reference translations) of Europarl-ST test set from the csv file generated in the experiment L4.1. Then, we prepare the test set in batches to be translated:

```
In [ ]: from datasets import load_dataset

raw_datasets = load_dataset("csv", data_files="L4.1_ASR_Whisper_Baseline_Europarl-ST.csv")

print(raw_datasets)
```

Each sample pair is preprocessed according to the training needs of the model that has been finetuned:

```
In [21]: def preprocess_function(sample):
          model_inputs = tokenizer(
              sample["hypothesis"],
              text_target = sample["translation"],
          )
          return model_inputs
```

```
In [ ]: tokenized_datasets = raw_datasets.map(preprocess_function, batched=True)
```

```
In [ ]: test_batch_size = 32
        batch_tokenized_test = tokenized_datasets['train'].batch(test_batch_size)
```

Processing in batches to add padding and converting to tensors, then perform inference with num_beams = 1 and do_sample = False, that is, greedy search.

```
In [ ]: number_of_batches = len(batch_tokenized_test["hypothesis"])
        output_sequences = []
        for i in range(number_of_batches):
            inputs = tokenizer(batch_tokenized_test["hypothesis"][i], max_length=max_tok_length, truncation=False, return_tensors='pt')
            output_batch = lora_model.generate(generation_config=generation_config, input_ids=inputs["input_ids"].cuda(), attn_implementation='flash_attention_2')
            output_sequences.extend(output_batch.cpu())
```

We can recover the decoded predictions and references by applying `batch_decode` of the tokenizer

```
In [25]: decoded_preds = tokenizer.batch_decode(output_sequences, skip_special_tokens=True)
```

```
In [26]: references = tokenizer.batch_decode(tokenized_datasets["train"]["labels"], skip_special_tokens=True)
```

Let's take a closer look at some decoded predictions and references:

```
In [ ]: references[:5]
```

```
In [ ]: decoded_preds[:5]
```

Predictions and references are normalized using the Whisper basic text standardisation/normalization module

```
In [29]: from whisper.normalizers.basic import BasicTextNormalizer
        normalizer = BasicTextNormalizer()

        decoded_preds_clean = [normalizer(text) for text in decoded_preds]
        references_clean = [normalizer(text) for text in references]
```

```
In [30]: from evaluate import load
        metric = load("sacrebleu")
```

For evaluation, we use the [Evaluate library](#) which includes the definition of generic and task-specific metrics. In our case, we use the [BLEU metric](#), or to be more precise, [sacreBLEU](#).

```
In [35]: result = metric.compute(predictions=decoded_preds_clean, references=references_clean)
print(f'BLEU score: {result["score"]:.1f}')
```

BLEU score: 25.4

```
In [ ]: from evaluate import load
comet_metric = load('comet')
```

Compute COMET figures using the [Evaluate library](#) which includes the definition of generic and task-specific metrics.

```
In [ ]: comet_score = comet_metric.compute(predictions=decoded_preds_clean, references=references_clean, sources=raw_dataset)
```

```
In [34]: print(f"COMET: {comet_score['mean_score'] * 100:.2f} %")
```

COMET: 70.09 %

As a result of the fine tuning, we have achieved a relative improvement of 173% in terms of BLEU (from 9.3 to 25.4) and about 21% in terms of COMET (from 57.9% to 70.1%)

Exercise

Perform a similar experiment using the Covost2 source-english setup previously used in L4.1. Evaluate the performance using the best whisper models obtained in L4.1.